# AP 12: REPLICATION – HAPROXY - HEARTBEAT

Bchi Aliya- Goodman Mathéo

lien Gantt :
https://docs.google.com/spreadsheets/d/1rpj3_1ULYG4NzPqijX1-0HSkOxV1oPCu-S4woIjAjYs/edit?usp=sharing

lien Schéma réseaux :
https://drive.google.com/file/d/1OUvV3dj2cvojEKEhfht_rPbyXuJPmcgI/view?usp=drive_link

## Mission 0 : Prérequis et contraintes

Les machines que nous avons créées sont les suivantes : **SRVmdb**, dédié à MariaDB ; **web1**, **web2** et **web3**, utilisés comme serveurs web ; **haproxy**, configuré comme serveur de répartition de charge ; et **CLTwebtest**, une machine cliente destinée aux tests.

Web 1 -> vlan 452

```
Network Device (net0)        virtio=BC:24:11:00:BA:A2,bridge=vmbr0,tag=452
```

La vm se nomme web1.

```
root@web1:~# _
```

Elle a pour Ip 192.168.12.20.

```
ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether bc:24:11:00:ba:a2 brd ff:ff:ff:ff:ff:ff
 altname enp0s18
 altname enxbc241100baa2
 inet 192.168.12.20/24 brd 192.168.12.255 scope global ens18
    valid_lft forever preferred_lft forever
 inet6 fe80::be24:11ff:fe00:baa2/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
```

Référencement DNS effectué pour notre machine afin d'associer son nom à son adresse IP.

```
web1    IN      A       192.168.12.20
```

Vérification du DNS réalisée avec **nslookup** pour confirmer la bonne résolution de ma machine.

```
root@clonedns:~# nslookup web1
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   web1.menuimetal.fr
Address: 192.168.12.20
```

Référencement de la machine dans **GLPI** afin d'assurer son inventaire et son suivi dans la gestion du parc.

| | | | | | | |
|---|---|---|---|---|---|---|
| web1 | Entité racine | QEMU | QEMU | Standard PC (i440FX + PIIX, 1996) | Debian GNU/ Linux 13 (trixie) | 2025-12-02 14:46 |

Page HTML créée pour être affichée dans un navigateur.

```
root@web1:~# echo "<html><body><h1>Je suis le serveur web1</h1></body></html>" | tee /var/w
ww/html/index.html
<html><body><h1>Je suis le serveur web1</h1></body></html>
root@web1:~#
```

web 2 -> vlan 452 (DMZ):

2129 (web2)

Network Device (net0) virtio=BC:24:11:29:3A:57,bridge=vmbr0,tag=452

La vm se nomme web2.

```
127.0.0.1       localhost
192.168.12.21   web2
```

L'Ip de la vm est 192.168.12.21.

```
ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
  link/ether bc:24:11:29:3a:57 brd ff:ff:ff:ff:ff:ff
  altname enp0s18
  altname enxbc2411293a57
  inet 192.168.12.21/24 brd 192.168.12.255 scope global ens18
     valid_lft forever preferred_lft forever
  inet6 fe80::be24:11ff:fe29:3a57/64 scope link proto kernel_ll
     valid_lft forever preferred_lft forever
root@web2:~#
```

Référencement DNS effectué pour notre machine afin d'associer son nom à son adresse IP.

```
web2    IN    A    192.168.12.21
```

Vérification du DNS réalisée avec **nslookup** pour confirmer la bonne résolution de ma machine.

```
root@clonedns:~# nslookup web2
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   web2.menuimetal.fr
Address: 192.168.12.21

root@clonedns:~#
```

Référencement de la machine dans **GLPI** afin d'assurer son inventaire et son suivi dans la gestion du parc.

| | | | | | | |
|---|---|---|---|---|---|---|
| web2 | Entité racine | QEMU | QEMU | Standard PC (i440FX + PIIX, 1996) | Debian GNU/ Linux 13 (trixie) | 2025-12-02 14:48 |

Page HTML créée pour être affichée dans un navigateur.

```
root@web2:~# echo "<html><body><h1>Je suis le serveur web2</h1></body></html>" | tee /var/w
ww/html/index.html
<html><body><h1>Je suis le serveur web2</h1></body></html>
root@web2:~#
```

Web 3 -> vlan 452

```
Network Device (net0)        virtio=BC:24:11:48:84:74,bridge=vmbr0,tag=452
```

Le nom de la vm est web3.

```
root@web3:~#
```

L'ip de la vm est 192.168.12.22.

```
ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether bc:24:11:48:84:74 brd ff:ff:ff:ff:ff:ff
 altname enp0s18
 altname enxbc2411488474
 inet 192.168.12.22/24 brd 192.168.12.255 scope global ens18
    valid_lft forever preferred_lft forever
 inet6 fe80::be24:11ff:fe48:8474/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
root@web3:~# _
```

Référencement DNS effectué pour notre machine afin d'associer son nom à son adresse IP.

```
web3    IN    A    192.168.12.22
```

Vérification du DNS réalisée avec **nslookup** pour confirmer la bonne résolution de ma machine.

```
root@clonedns:~# nslookup web3
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   web3.menuimetal.fr
Address: 192.168.12.22
```

Référencement de la machine dans **GLPI** afin d'assurer son inventaire et son suivi dans la gestion du parc.

| | | | | | | |
|---|---|---|---|---|---|---|
| web3 | Entité racine | QEMU | QEMU | Standard PC (i440FX + PIIX, 1996) | Debian GNU/ Linux 13 (trixie) | 2025-12-02 14:52 |

Page HTML créée pour être affichée dans un navigateur.

```
root@web3:~# echo "<html><body><h1>Je suis le serveur web3</h1></body></html>" | tee /var/w
ww/html/index.html
<html><body><h1>Je suis le serveur web3</h1></body></html>
root@web3:~#
```

Haproxy -> Vlan 452

```
Network Device (net0)        virtio=BC:24:11:36:73:D2,bridge=vmbr0,tag=452
```

Le nom de la vm est Haproxy.

```
root@Haproxy:~# _
```

L'ip de la vm est 192.168.12.26.

```
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:36:73:d2 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24113673d2
    inet 192.168.12.26/24 brd 192.168.12.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe36:73d2/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
sio@Haproxy:~$
```

Référencement DNS effectué pour notre machine afin d'associer son nom à son adresse IP.

```
Haproxy IN      A       192.168.12.26
```

Vérification du DNS réalisée avec **nslookup** pour confirmer la bonne résolution de ma machine.

```
root@clonedns:~# nslookup Haproxy
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   Haproxy.menuimetal.fr
Address: 192.168.12.26
```

Référencement de la machine dans **GLPI** afin d'assurer son inventaire et son suivi dans la gestion du parc.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Haproxy | Entité racine | QEMU | QEMU | Standard PC (i440FX + PIIX, 1996) | Debian GNU/ Linux 13 (trixie) | 2025-12-02 14:35 | |

CLTwebtest -> Vlan 452

| | |
|---|---|
| Network Device (net0) | virtio=BC:24:11:5E:89:51,bridge=vmbr0,tag=452 |

Le nom de notre cliente est CLTwebtest.

```
sio@CLTwebtest:~
```

L'ip de la vm est 192.168.12.27.

```
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether bc:24:11:5e:89:51 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24115e8951
    inet 192.168.12.27/24 brd 192.168.12.255 scope global noprefixroute ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::96f0:cf9d:75f1:33b2/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
sio@CLTwebtest:~$
```

Référencement de la machine dans **GLPI** afin d'assurer son inventaire et son suivi dans la gestion du parc.

| | | | | | | |
|---|---|---|---|---|---|---|
| CLTwebtest | Entité racine | QEMU | QEMU | Standard PC (i440FX + PIIX, 1996) | Debian GNU/ Linux 13 (trixie) | 2025-12-02 14:42 |

SRVmdb -> vlan 452

| | |
|---|---|
| Network Device (net0) | virtio=BC:24:11:70:AD:FC,bridge=vmbr0,tag=451 |

Le nom de la vm mariadb est SRVmdb2.

```
192.168.11.30    SRVmdb2
```

L'ip de la vm est 192.168.11.30.

```
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
en 1000
    link/ether bc:24:11:70:ad:fc brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc241170adfc
    inet 192.168.11.30/24 brd 192.168.11.255 scope global e
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe70:adfc/64 scope link proto ker
       valid_lft forever preferred_lft forever
```

Référencement DNS effectué pour notre machine afin d'associer son nom à son adresse IP.

```
SRVmdb2 IN      A       192.168.11.30
```

Vérification du DNS réalisée avec **nslookup** pour confirmer la bonne résolution de ma machine.

```
root@clonedns:~# nslookup SRVmdb2
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   SRVmdb2.menuimetal.fr
Address: 192.168.11.30

root@clonedns:~#
```

Référencement de la machine dans **GLPI** afin d'assurer son inventaire et son suivi dans la gestion du parc.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| SRVmdb | Entité racine | QEMU | QEMU | Standard PC (i440FX + PIIX, 1996) | Debian GNU/ Linux 13 (trixie) | 2025-12-02 15:10 | |

## Mission 1 : RÉPLICATION DU SERVEUR MYSQL:

**Pourquoi Monsieur Lepage a intérêt à mettre en place la réplication?**

->Grâce à la réplication, les données présentes sur le serveur maître sont automatiquement copiées vers un serveur secondaire, ce qui permet de disposer d'une copie toujours à jour. En cas de panne du serveur principal

**Qu'est-ce qu'une réplication?**

- >La réplication MySQL est un mécanisme qui permet de dupliquer automatiquement les données d'un serveur maître vers un serveur esclave

**Quel résultat doit être attendu?**

->on doit s'attendre à ce que toutes les modifications effectuées sur le serveur maître — qu'il s'agisse de la création d'une base de données, d'une table ou d'une simple insertion de données ,soient automatiquement et rapidement répercutées sur le serveur slave.

*CONFIGURATION :*

Les étapes pour mettre en place la réplication Master/Slave.

Création d'un utilisateur MySQL pour permettre l'accès à la base de données.

```
root@SRVmdb:~# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 39
Server version: 10.3.29-MariaDB-0+deb10u1-log Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Après m'être connecté en SSH, nous sommes allés dans le fichier de configuration de MariaDB (`/etc/mysql/mariadb.conf.d/50-server.cnf`) sur le serveur maître, qui a le rôle de serveur de base de données. Ce serveur maître a pour fonction de gérer la réplication.

```
#
# * Basic Settings
#
user                    = mysql
pid-file                = /run/mysqld/mysqld.pid
socket                  = /run/mysqld/mysqld.sock
#port                   = 3306
basedir                 = /usr
datadir                 = /var/lib/mysql
tmpdir                  = /tmp
lc-messages-dir         = /usr/share/mysql
#skip-external-locking

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address            = 0.0.0.0
server-id       = 1
log_bin = /var/log/mysql/mariadb-bin
#
```

Configuration du serveur esclave de MariaDB : il a pour rôle de relayer les données de la réplication.

```
#
# * Basic Settings
#
user                    = mysql
pid-file                = /run/mysqld/mysqld.pid
socket                  = /run/mysqld/mysqld.sock
#port                   = 3306
basedir                 = /usr
datadir                 = /var/lib/mysql
tmpdir                  = /tmp
lc-messages-dir         = /usr/share/mysql
#skip-external-locking

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address            = 0.0.0.0
server-id       = 2
relay_log       = /var/log/mysql/relay-bin
#
```

Sauvegarde de toutes les bases de données du compte root sur le serveur MariaDB maître.

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 41
Server version: 10.3.29-MariaDB-0+deb10u1-log Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW MASTER STATUS;
+--------------------+----------+--------------+------------------+
| File               | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+--------------------+----------+--------------+------------------+
| mariadb-bin.000002 |      836 |              |                  |
+--------------------+----------+--------------+------------------+
1 row in set (0.000 sec)

MariaDB [(none)]> scp dump_master.sql root@192.168.12.30:/root/
    -> Ctrl-C -- exit!
Aborted
```

Ici, nous pouvons consulter toutes les données contenues dans les tables.Transfert des bases de données de notre serveur maître vers notre serveur esclave à l'aide de la commande :`scp dump_master.sql root@192.168.12.30:`

```
root@mariadb:~# scp dump_master.sql root@192.168.12.30:/root/
The authenticity of host '192.168.12.30 (192.168.12.30)' can't be established.
ED25519 key fingerprint is SHA256:Klmjaw4o9Bb97rdBXE/IEnXucRm0hkvF/lW53X/ccgk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.12.30' (ED25519) to the list of known hosts.
root@192.168.12.30's password:
Permission denied, please try again.
root@192.168.12.30's password:
dump_master.sql                                       100%  466KB  26.5MB/s   00:00
root@mariadb:~#
```

Importation des données sur le serveur esclave, puis connexion à MySQL.

```
root@SRVmdb:~# mysql -u root -p < /root/dump_master.sql
Enter password:
root@SRVmdb:~# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 39
Server version: 10.3.29-MariaDB-0+deb10u1-log Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Configuration de la réplication sur le serveur esclave.

```
MariaDB [(none)]> CHANGE MASTER TO
    ->   MASTER_HOST='192.168.11.1',
    ->   MASTER_USER='sio',
    ->   MASTER_PASSWORD='sio',
    ->   MASTER_LOG_FILE='mariadb-bin.000002',
    ->   MASTER_LOG_POS=2522;
Query OK, 0 rows affected (0.016 sec)

MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0.001 sec)
```

Statut de la réplication

```
MariaDB [(none)]> SHOW SLAVE STATUS\G
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: 192.168.11.1
                  Master_User: sio
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mariadb-bin.000006
          Read_Master_Log_Pos: 344
               Relay_Log_File: slave-relay-bin.000009
                Relay_Log_Pos: 645
        Relay_Master_Log_File: mariadb-bin.000006
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
          Replicate_Ignore_DB:
           Replicate_Do_Table:
       Replicate_Ignore_Table:
      Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:
                   Last_Errno: 0
                   Last_Error:
                 Skip_Counter: 0
          Exec_Master_Log_Pos: 344
              Relay_Log_Space: 1255
              Until_Condition: None
               Until_Log_File:
                Until_Log_Pos: 0
           Master_SSL_Allowed: No
           Master_SSL_CA_File:
           Master_SSL_CA_Path:
              Master_SSL_Cert:
            Master_SSL_Cipher:
               Master_SSL_Key:
        Seconds_Behind_Master: 0
 Master_SSL_Verify_Server_Cert: No
                Last_IO_Errno: 0
                Last_IO_Error:
```

J'ai affiché les bases de données disponibles et répliquées sur mon serveur esclave.

```
MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| glpi               |
| information_schema |
| mysql              |
| performance_schema |
| test_matheo        |
| test_replica       |
| test_replication   |
| test_sio           |
| test_sio21         |
| testdb             |
+--------------------+
10 rows in set (0.001 sec)

MariaDB [(none)]> USE test_replication
Database changed
MariaDB [test_replication]> SHOW TABLES
    -> Ctrl-C -- exit!
Aborted
root@SRVmdb:~# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 40
Server version: 10.3.29-MariaDB-0+deb10u1-log Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
```

Utilisation de la base de données répliquée sur le serveur esclave.

```
MariaDB [(none)]> USE test_matheo
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [test_matheo]> SHOW TABLES;
+-----------------------+
| Tables_in_test_matheo |
+-----------------------+
| table1                |
| table2                |
+-----------------------+
2 rows in set (0.000 sec)

MariaDB [test_matheo]> SELECT * FROM table1;
+----+-------+
| id | nom   |
+----+-------+
|  1 | Alice |
+----+-------+
1 row in set (0.000 sec)

MariaDB [test_matheo]> SELECT * FROM table2;
+----+--------+
| id | valeur |
+----+--------+
|  1 |    100 |
+----+--------+
1 row in set (0.000 sec)

MariaDB [test_matheo]>
```

on peut afficher les tables disponibles  et aller sur la tables .

Voici les logs affichés pour prouver que la réplication et les connexions à la base de données ont bien été effectuées, dans le fichier `/var/log/mysql/error.log`.

```
se '--log-basename=#' or '--relay-log=mysqld-relay-bin' to avoid this problem.
2025-12-04 15:04:29 10 [Note] Slave I/O thread: Start asynchronous replication to master 'sio@
192.168.11.1:3306' in log 'mariadb-bin.00000X' at position 12345
2025-12-04 15:04:29 11 [Note] Slave SQL thread initialized, starting replication in log 'maria
db-bin.00000X' at position 12345, relay log './mysqld-relay-bin.000004' position: 4
2025-12-04 15:04:29 0 [Note] /usr/sbin/mysqld: ready for connections.
Version: '10.3.29-MariaDB-0+deb10u1-log'  socket: '/run/mysqld/mysqld.sock'  port: 3306  Debia
n 10
2025-12-04 15:04:29 10 [Note] Slave I/O thread: connected to master 'sio@192.168.11.1:3306',re
plication started in log 'mariadb-bin.00000X' at position 12345
2025-12-04 15:04:29 10 [ERROR] Slave I/O: Fatal error: The slave I/O thread stops because mast
er and slave have equal MySQL server ids; these ids must be different for replication to work
(or the --replicate-same-server-id option must be used on slave but this does not always make
sense; please check the manual before using it). Internal MariaDB error code: 1593
2025-12-04 15:04:29 10 [Note] Slave I/O thread exiting, read up to log 'mariadb-bin.00000X', p
osition 12345
2025-12-04 15:04:29 10 [Note] master was 192.168.11.1:3306
2025-12-04 15:05:27 41 [Warning] IP address '192.168.12.30' could not be resolved: Name or ser
vice not known
2025-12-04 15:05:27 41 [Note] Start binlog_dump to slave_server(2), pos(mariadb-bin.000005, 34
4), using_gtid(0), gtid('')
```

Nous avons modifié le fichier de configuration des logs des serveurs MariaDB (`/etc/rsyslog.conf`) afin que les logs soient directement envoyés vers notre fichier de logs sur le serveur Rsyslog.
(serveur maître)

```
$IncludeConfig /etc/rsyslog.d/*.conf


###############
#### RULES ####
###############

#
# Log anything besides private authentication messages to a single log file
#
*.*;auth,authpriv.none          -/var/log/syslog

#
# Log commonly used facilities to their own log file
#
auth,authpriv.*                 /var/log/auth.log
cron.*                          -/var/log/cron.log
kern.*                          -/var/log/kern.log
mail.*                          -/var/log/mail.log
user.*                          -/var/log/user.log

#
# Emergencies are sent to everybody logged in.
#
*.emerg                         :omusrmsg:*

*.* 192.168.13.5                :514
"/etc/rsyslog.conf" 71L, 1468B                                  71,33         Bot
```

(Serveur esclave) : j'ai ajouté le double `@@` pour utiliser le protocole TCP.

```
#### RULES ####
###############

#
# Log anything besides private authentication messages to a single log file
#
*.*;auth,authpriv.none          -/var/log/syslog

#
# Log commonly used facilities to their own log file
#
auth,authpriv.*                 /var/log/auth.log
cron.*                          -/var/log/cron.log
kern.*                          -/var/log/kern.log
mail.*                          -/var/log/mail.log
user.*                          -/var/log/user.log

#
# Emergencies are sent to everybody logged in.
#
*.emerg                         :omusrmsg:*

*.* @@192.168.13.5:514

module(load="imtcp")
input(type="imtcp" port="514")

"/etc/rsyslog.conf" 90L, 1719B                                  86,5          Bot
```

Puis, j'ai effectué un `systemctl restart rsyslog`.

Petit test pour vérifier si les logs de Rsyslog fonctionnent correctement :
Sur la machine esclave, on exécute la commande suivante :

```
root@SRVmdb:~# logger "TEST_RSYSLOG depuis client"
root@SRVmdb:~# vim /etc/rsyslog.conf
root@SRVmdb:~# logger "TEST_RSYSLOG depuis client"
root@SRVmdb:~# logger "TEST_RSYSLOG depuis client2 "
root@SRVmdb:~#
```

Ensuite, nous consultons Rsyslog pour observer les événements en temps réel avec la commande :`tail -f /var/log/syslog`

```
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: imfile: on startup file '/var/log/mysql/mari
adb.log' does not exist but is configured in static file monitor - this may indicate a
misconfiguration. If the file appears at a later time, it will automatically be process
ed. Reason: No such file or directory [v8.2504.0]
2025-12-04T15:52:15+01:00 SRVmdb systemd[1]: Started rsyslog.service - System Logging S
ervice.
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: [origin software="rsyslogd" swVersion="8.250
4.0" x-pid="5971" x-info="https://www.rsyslog.com"] start
2025-12-04T15:53:32+01:00 SRVmdb root: TEST_RSYSLOG depuis client
2025-12-04T15:53:32+01:00 SRVmdb root: TEST_RSYSLOG depuis client
2025-12-04T16:02:02+01:00 SRVmdb root: TEST_RSYSLOG depuis client
2025-12-04T16:02:02+01:00 SRVmdb root: TEST_RSYSLOG depuis client
2025-12-04T16:02:14+01:00 SRVmdb root: TEST_RSYSLOG depuis client2
2025-12-04T16:02:14+01:00 SRVmdb root: TEST_RSYSLOG depuis client2
```

**Test des logs :**
Nous avons vérifié que les logs des serveurs MariaDB fonctionnaient correctement et que les traces des serveurs étaient bien visibles. J'ai pu effectuer un test en utilisant la commande `logger` depuis ma machine cliente.

```
root@SrvRsyslog:~# tail -f /var/log/syslog
2025-12-04T15:52:15+01:00 SRVmdb systemd[1]: Stopped rsyslog.service - System Logging S
ervice.
2025-12-04T15:52:15+01:00 SRVmdb systemd[1]: Starting rsyslog.service - System Logging
Service...
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: imuxsock: Acquired UNIX socket '/run/systemd
/journal/syslog' (fd 3) from systemd.  [v8.2504.0]
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: imfile: on startup file '/var/log/mysql/mari
adb.log' does not exist but is configured in static file monitor - this may indicate a
misconfiguration. If the file appears at a later time, it will automatically be process
ed. Reason: No such file or directory [v8.2504.0]
2025-12-04T15:52:15+01:00 SRVmdb systemd[1]: Started rsyslog.service - System Logging S
ervice.
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: [origin software="rsyslogd" swVersion="8.250
4.0" x-pid="5971" x-info="https://www.rsyslog.com"] start
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: imuxsock: Acquired UNIX socket '/run/systemd
/journal/syslog' (fd 3) from systemd.  [v8.2504.0]
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: imfile: on startup file '/var/log/mysql/mari
adb.log' does not exist but is configured in static file monitor - this may indicate a
misconfiguration. If the file appears at a later time, it will automatically be process
ed. Reason: No such file or directory [v8.2504.0]
2025-12-04T15:52:15+01:00 SRVmdb systemd[1]: Started rsyslog.service - System Logging S
ervice.
2025-12-04T15:52:15+01:00 SRVmdb rsyslogd: [origin software="rsyslogd" swVersion="8.250
4.0" x-pid="5971" x-info="https://www.rsyslog.com"] start
2025-12-04T15:53:32+01:00 SRVmdb root: TEST_RSYSLOG depuis client
2025-12-04T15:53:32+01:00 SRVmdb root: TEST_RSYSLOG depuis client
```

Nous avons envoyé des tests pour vérifié que les logs de la machine esclave apparaissaient correctement sur Rsyslog.

```
root@SRVmdb:~# logger "TEST_RSYSLOG depuis client4 "

root@SRVmdb:~# logger "TEST_RSYSLOG depuis client4 "
root@SRVmdb:~# logger "TEST_RSYSLOG depuis client5 "
root@SRVmdb:~#
```

Dans le fichier de logs (`tail -f /var/log/syslog`) sur Rsyslog, nous pouvons voir les logs provenant du serveur esclave.

```
2025-12-05T08:33:49+01:00 SRVmdb qemu-ga: info: guest-ping called
2025-12-05T08:33:49+01:00 SRVmdb qemu-ga: info: guest-ping called
2025-12-05T08:34:03+01:00 SRVmdb qemu-ga: info: guest-ping called
2025-12-05T08:34:03+01:00 SRVmdb qemu-ga: info: guest-ping called
2025-12-05T08:34:13+01:00 SRVmdb root: TEST_RSYSLOG depuis client4
2025-12-05T08:34:13+01:00 SRVmdb root: TEST_RSYSLOG depuis client4
2025-12-05T08:34:17+01:00 SRVmdb qemu-ga: info: guest-ping called
2025-12-05T08:34:17+01:00 SRVmdb qemu-ga: info: guest-ping called
2025-12-05T08:34:18+01:00 SRVmdb root: TEST_RSYSLOG depuis client5
2025-12-05T08:34:18+01:00 SRVmdb root: TEST_RSYSLOG depuis client5
```

Voici une petite procédure pour vérifier la configuration des logs des serveurs maître et esclave :

Sur cette capture, nous avons vérifié si le fichier général des logs était activé. Si ce n'est pas le cas, il faut l'activer dans le fichier :`/var/log/mysql/mariadb.log`

```
MariaDB [(none)]> SHOW VARIABLES LIKE 'general_log';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| general_log   | ON    |
+---------------+-------+
1 row in set (0.001 sec)

MariaDB [(none)]> SHOW VARIABLES LIKE 'general_log_file';
+-----------------+---------------------------+
| Variable_name   | Value                     |
+-----------------+---------------------------+
| general_log_file | /var/log/mysql/mariadb.log |
+-----------------+---------------------------+
1 row in set (0.001 sec)

MariaDB [(none)]> USE sio;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

```
$ModLoad imfile

# Surveiller le fichier de logs MariaDB (error log)
input(type="imfile"
      File="/var/log/mysql/mariadb.log"
      Tag="mariadb-master:"
      Severity="info"
      Facility="local6")

# Envoyer vers ton serveur Rsyslog central
local6.* @@192.168.13.5:514
```

La capture de droite montre le fichier que j'ai créé afin que tous les logs soient bien structurés et enregistrés dans le fichier :`/var/log/mysql/mariadb.log`

Cela nous permet de vérifier que Rsyslog reçoit correctement les logs du serveur maître.

Pour tester si les logs sont bien enregistrés sur la machine maître dans le fichier général des logs, nous pouvons utiliser la commande :`tail -f /var/log/mysql/mariadb.log`

```
Database changed
MariaDB [sio]> CREATE TABLE test_rsyslog (id INT PRIMARY KEY, name VARCHAR(50));
Query OK, 0 rows affected (0.070 sec)

MariaDB [sio]> tail -f /var/log/mysql/general.log
    -> CCtrl-C -- exit!
Aborted
root@mariadb:~# tail -f /var/log/mysql/
error.log           error.log.5.gz      mariadb-bin.000003  mariadb-bin.index
error.log.1.gz      error.log.6.gz      mariadb-bin.000004  mariadb.log
error.log.2.gz      error.log.7.gz      mariadb-bin.000005
error.log.3.gz      mariadb-bin.000001  mariadb-bin.000006
error.log.4.gz      mariadb-bin.000002  mariadb-bin.000007
root@mariadb:~# tail -f /var/log/mysql/mariadb.log
                    43 Query    select @@version_comment limit 1
251205 10:56:58     43 Query    SHOW VARIABLES LIKE 'general_log'
                    43 Query    SHOW VARIABLES LIKE 'general_log_file'
251205 10:57:31     43 Query    SELECT DATABASE()
                    43 Init DB  sio
                    43 Query    show databases
                    43 Query    show tables
                    43 Field List       test_sio1
251205 10:57:52     43 Query    CREATE TABLE test_rsyslog (id INT PRIMARY KEY, name VARCHAR(50
))
251205 10:58:06     43 Quit
```

Voici le résultat des logs de la machine maître, qui enregistrent des actions telles que la création de bases de données ou la création de tables dans les bases de données.
On peut clairement voir le nom de la machine, si le serveur est le maître ou non, ainsi que le moment où les modifications ont été effectuées.

```
root@SrvRsyslog:~# tail -f /var/log/mariadb/mariadb.log
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Field List#011test_rsyslog
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Field List#011test_sio1
2025-12-05T11:16:05+01:00 mariadb mariadb-master: 251205 11:15:56#011    44 Query#011SELECT DA
TABASE()
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Init DB#011sio
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Query#011show databases
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Query#011show tables
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Field List#011test_rsyslog
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Field List#011test_sio1
2025-12-05T11:16:15+01:00 mariadb mariadb-master: 251205 11:16:09#011    44 Query#011CREATE TA
BLE test_siobo (id INT PRIMARY KEY, name VARCHAR(50))
2025-12-05T11:16:15+01:00 mariadb mariadb-master: 251205 11:16:09#011    44 Query#011CREATE TA
BLE test_siobo (id INT PRIMARY KEY, name VARCHAR(50))
^C
```

Ensuite, nous avons retesté avec une autre table afin de valider que la configuration fonctionne correctement.

```
root@mariadb:~# vim /etc/rsyslog.d/mariadb.conf
root@mariadb:~# systemctl restart rsyslog
root@mariadb:~# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 44
Server version: 10.3.29-MariaDB-0+deb10u1-log Debian 10

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE sio
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [sio]> CREATE TABLE test_siobo (id INT PRIMARY KEY, name VARCHAR(50));
Query OK, 0 rows affected (0.216 sec)
```

et voici le résultat sur Rsyslog :

```
2025-12-05T11:16:05+01:00 mariadb mariadb-master: #011#011    44 Field List#011test_sio1
2025-12-05T11:16:15+01:00 mariadb mariadb-master: 251205 11:16:09#011    44 Query#011CREATE TA
BLE test_siobo (id INT PRIMARY KEY, name VARCHAR(50))
2025-12-05T11:16:15+01:00 mariadb mariadb-master: 251205 11:16:09#011    44 Query#011CREATE TA
BLE test_siobo (id INT PRIMARY KEY, name VARCHAR(50))
```

En créant ce fichier **`/etc/rsyslog/mariadb-slave.conf`**, le serveur esclave pourra conserver les logs des actions de réplication effectuées par le serveur maître.

```
$ModLoad imfile

input(type="imfile"
      File="/var/log/mysql/mysql.log"
      Tag="mariadb-slave:"
      Severity="info"
      Facility="local6")

local6.* @@192.168.13.5:514
```

Les logs sont envoyés vers **`/var/log/mysql/mysql.log`**, ce qui permet à Rsyslog de récupérer correctement les logs de réplication et des actions effectuées. Voici les résultats.

## Mission 2 : Mise en place d'une répartition de charges sur un serveur Web avec HaProxy

***Quel est l'intérêt d'une répartition de charges ?**

-> La répartition de charges permet de partager le trafic entre plusieurs serveurs afin d'éviter la surcharge d'un seul, d'améliorer les performances du service et d'assurer une meilleure disponibilité. Si un serveur tombe en panne, les autres prennent automatiquement le relais, ce qui garantit la continuité du service.

***Mettre en place la répartition de charges à l'aide de l'applicatif HaProxy.**

Pour mettre en place la répartition de charges, il faut d'abord installer HAProxy sur la machine qui servira de load balancer.

```
root@Haproxy:~# apt install haproxy -y
```

Une fois installé, le service HAProxy peut être démarré ou redémarré avec et verifier le statut

```
root@Haproxy:~# systemctl restart haproxy
root@Haproxy:~# systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset: enabled)
     Active: active (running) since Tue 2025-12-02 15:02:26 CET; 8s ago
 Invocation: e183e96aa95f4e7ca89899c528ce572f
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
   Main PID: 2443 (haproxy)
     Status: "Ready."
```

**a. Quelles sont les sections que l'on trouve sur fichier de configuration haproxy.cfg, expliquer leur rôle.**

Le fichier de configuration contient plusieurs sections importantes.

- **global** : définit les paramètres globaux comme les logs ou le nombre de connexions.
- **defaults** : définit les réglages par défaut utilisés par les frontends et backends (timeouts, mode http…).
- **frontend** : point d'entrée du trafic ; il reçoit les requêtes des clients sur un port précis.
- **backend** : liste des serveurs web vers lesquels HAProxy envoie le trafic ainsi que la méthode de répartition.

Le fichier `haproxy.cfg` est incomplet, il lui manque les sections **frontend** et **backend**.

```
  GNU nano 8.4                    /etc/haproxy/haproxy.cfg
global
        log /dev/log    local0
        log /dev/log    local1 notice
        chroot /var/lib/haproxy
        stats socket /run/haproxy/admin.sock mode 660 level admin
        stats timeout 30s
        user haproxy
        group haproxy
        daemon

        # Default SSL material locations
        ca-base /etc/ssl/certs
        crt-base /etc/ssl/private

        # See: https://ssl-config.mozilla.org/#server=haproxy&server-version=2.0.3&config=>
        ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256>
        ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CH>
        ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
        log     global
        mode    http
        option  httplog
        option  dontlognull
        timeout connect 5000
        timeout client  50000
        timeout server  50000
        errorfile 400 /etc/haproxy/errors/400.http
        errorfile 403 /etc/haproxy/errors/403.http
        errorfile 408 /etc/haproxy/errors/408.http
        errorfile 500 /etc/haproxy/errors/500.http
        errorfile 502 /etc/haproxy/errors/502.http
        errorfile 503 /etc/haproxy/errors/503.http
        errorfile 504 /etc/haproxy/errors/504.http
```

On configure les sections **frontend** et **backend** dans le fichier `haproxy.cfg` et j'ai ajouté les trois serveurs web.

```
frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server web1 192.168.12.20:80 check
    server web2 192.168.12.21:80 check
    server web3 192.168.12.22:80 check
```

Le bloc **frontend** et **backend** est correct pour les trois serveurs web. Voici ce que cela fait :

- Le **frontend** écoute toutes les requêtes HTTP sur le port **80**.
- Toutes les requêtes reçues sont envoyées au **backend** nommé `http_back`.
- Le **backend** contient les trois serveurs (`web1`, `web2`, `web3`) avec leurs adresses IP.
- La **balance** `roundrobin` répartit les requêtes à tour de rôle entre les trois serveurs.
- L'option `check` permet à HAProxy de vérifier que chaque serveur est en ligne avant d'envoyer le trafic.

On vérifie le statut pour s'assurer que tout fonctionne correctement, et tout est bon, sans erreur.

***Tester le serveur en arrêtant un serveur web, pour voir si les autres prennent le relais**

On commence par arrêter un serveur, ici **web1**.

```
root@web1:~# systemctl stop apache2
root@web1:~# systemctl status apache2
○ apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: inactive (dead) since Tue 2025-12-02 15:55:01 CET; 15s ago
   Duration: 1h 45min 35.554s
 Invocation: 0b527ffaf2414d71969d0467aa273c4e
       Docs: https://httpd.apache.org/docs/2.4/
```

En effectuant un test, le serveur redirige désormais le trafic uniquement vers **web2** et **web3**.

Et si on réactive **web1**, HAProxy détectera automatiquement que le serveur est de nouveau **UP** et répartira à nouveau le trafic dessus.

Les tests de répartition de charge ont été concluants : HAProxy distribue correctement le trafic entre les serveurs et continue à fonctionner même lorsqu'un serveur est arrêté.

***Mettre en place des statistiques et commenter les résultats obtenus**

On va d'abord activer les statistiques dans le fichier **`haproxy.cfg`**. Pour cela, on ajoute ce bloc à la fin du fichier, après les sections **frontend** et **backend**.

```
listen stats
    bind *:8080
    stats enable
    stats uri /stats
    stats refresh 10s
    stats auth admin:password
```

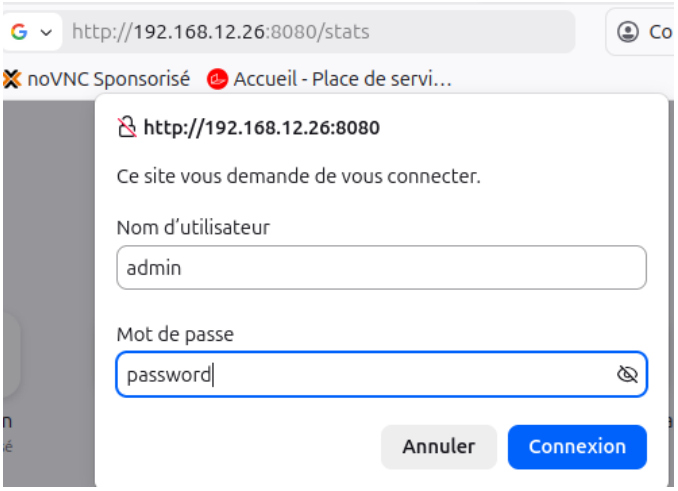
**Explications du bloc de statistiques HAProxy :**

- `bind *:8080` → définit le port sur lequel l'interface de statistiques sera accessible.
- `stats enable` → active les statistiques.
- `stats uri /stats` → définit l'URL pour consulter la page (ex. : `http://<IP_HAPROXY>:8080/stats`).
- `stats refresh 10s` → actualise la page toutes les 10 secondes.
- `stats auth admin:password` → configure le login et le mot de passe pour sécuriser l'accès.

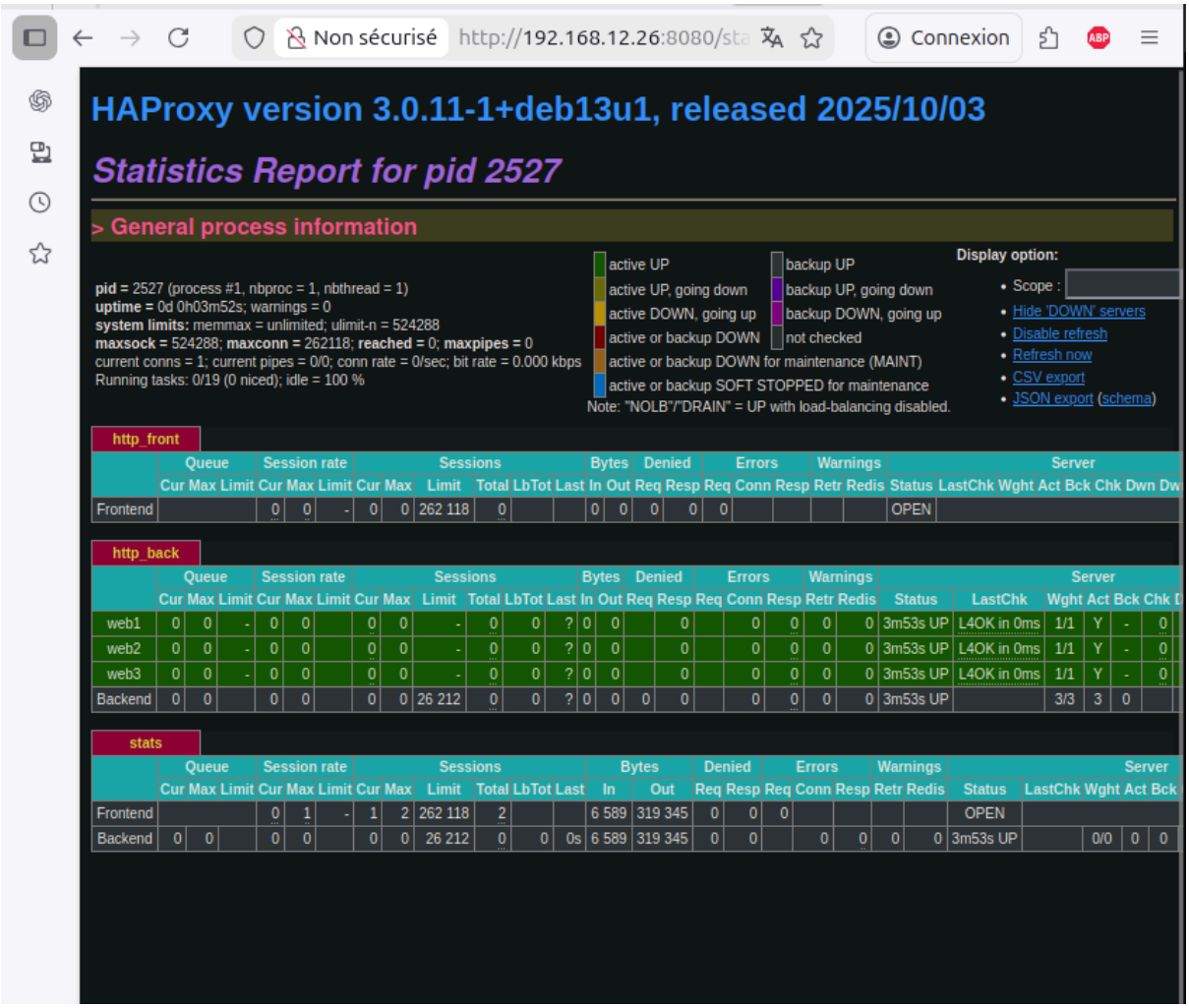
Puis on redémarre Haproxy.

```
root@Haproxy:~# haproxy -c -f /etc/haproxy/haproxy.cfg
root@Haproxy:~# systemctl restart haproxy
root@Haproxy:~#
```

Ensuite, on ouvre un navigateur et on saisit `http://192.168.12.26:8080/stats` pour accéder aux statistiques. Au préalable, il demande un mot de passe que l'on a configuré : **login : admin, mot de passe : password**.



Nous voici maintenant sur l'interface des statistiques.




Cette interface de statistiques HAProxy montre que les trois serveurs web sont bien en ligne et prêts à recevoir du trafic. Tous les serveurs sont en statut « UP », ce qui signifie qu'HAProxy les surveille activement et répartit les requêtes entre eux sans problème. Les données affichées confirment que le load balancing fonctionne correctement et que la solution assure une bonne disponibilité et tolérance aux pannes.

Je fais un test en arrêtant **web1** pour vérifier qu'il s'éteint correctement. Cela fonctionne : **web1** est **DOWN** et on peut voir les résultats en direct sur l'interface.

***Remonter HaProxy sur Nagios en supervisant les éléments suivants :**
**a. l'état de fonctionnement (allumé/éteint),**
**b. la charge du système,**
**c. l'état des interfaces réseau (up/down),**
**d. le taux de remplissage du disque dur,**
**e. l'état du service HaProxy et de ses stats**

On se connecte ensuite sur **Nagios**, on ouvre le fichier `localhost.cfg` et on y ajoute tous les `define` correspondant aux services à surveiller.

```
#HAProxy


define host{
    use                     linux-server
    host_name               Haproxy
    alias                   Serveur HAProxy
    address                 192.168.12.26
    max_check_attempts      5
    check_interval          5
    retry_interval          1
    check_period            24x7
    notification_interval   30
    notification_period     24x7
}

# Ping (état allumé/éteint)
define service{
    use                     generic-service
    host_name               Haproxy
    service_description     Ping
    check_command           check_ping!100.0,20%!500.0,60%
}

# Charge CPU
define service{
    use                     generic-service
    host_name               Haproxy
    service_description     Load Average
    check_command           check_load!5,4,3!10,6,4
}
```

```
# Interface réseau (ex: eth0)
define service{
    use                     generic-service
    host_name               Haproxy
    service_description     Interface eth0 Status
    check_command           check_ifoperstatus!-i eth0
}

# Disque dur
define service{
    use                     generic-service
    host_name               Haproxy
    service_description     Disk Usage
    check_command           check_disk!-w 20% -c 10% -p /
}

# Service HAProxy
define service{
    use                     generic-service
    host_name               Haproxy
    service_description     HAProxy Service
    check_command           check_procs!-C haproxy -c 1:
}

# Backend HAProxy / stats
define service{
    use                     generic-service
    host_name               Haproxy
    service_description     HAProxy Backend Health
    check_command           check_haproxy_stats!-s /run/haproxy/admin.sock
}
```

Ensuite, dans le fichier `commands.cfg`, on ajoute les **commandes spécifiques** correspondant aux services à surveiller.

```
#haproxy
# Vérifier l'état d'une interface réseau (modifier eth0 si nécessaire)
define command{
    command_name    check_ifoperstatus
    command_line    /usr/lib/nagios/plugins/check_ifoperstatus -H $HOSTADDRESS$ ->
}

# Vérifier que le service HAProxy tourne
define command{
    command_name    check_haproxy_service
    command_line    /usr/lib/nagios/plugins/check_procs -C haproxy -c 1:
}

# Vérifier la santé des backends HAProxy via le socket stats
define command{
    command_name    check_haproxy_stats
    command_line    /usr/lib/nagios/plugins/check_haproxy -s /run/haproxy/admin.s>
}
```

Je lance cette commande pour vérifier s'il y a des erreurs dans tous les fichiers de configuration inclus (**hosts**, **services**, **commandes**) et pour signaler d'éventuelles erreurs.

```
root@Nagios1:~# nagios4 -v /etc/nagios4/nagios.cfg
```

Et il n'y a **aucune erreur**, donc tout fonctionne correctement.

```
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0
```

On redémarre **Nagios** pour appliquer les changements.

```
root@Nagios1:~# systemctl restart nagios4
root@Nagios1:~# systemctl status nagios4
● nagios4.service - nagios4
     Loaded: loaded (/usr/lib/systemd/system/nagios4.service; enabled; preset: en>
     Active: active (running) since Tue 2025-12-02 16:43:32 CET; 6s ago
 Invocation: 178d18d8da864d578d0bfc54e0902a52
```

Voici le résultat obtenu sur **Nagios** après l'application des modifications :

| | | | | | | |
|---|---|---|---|---|---|---|
| Haproxy | CPU Load | OK | 12-05-2025 16:41:10 | 0d 5h 47m 35s | 1/3 | LOAD OK - total load average: 0.00, 0.01, 0.00 |
| | DISK | OK | 12-05-2025 16:39:51 | 0d 5h 43m 42s | 1/3 | DISK OK - free space: / 26904MiB (91,2% inode=96%): |
| | HAProxy Service | OK | 12-05-2025 16:37:22 | 2d 23h 56m 11s | 1/3 | PROCS OK: 104 processes |
| | HAProxy Stats | UNKNOWN | 12-05-2025 16:41:28 | 0d 0h 2m 5s | 3/3 | Usage: check_haproxy [ -v |
| | Heartbeat Status | OK | 12-05-2025 16:36:06 | 0d 2h 27m 27s | 1/3 | TCP OK - 0,001 second response time on 192.168.12.26 port 8080 |
| | Interface eth0 Status | UNKNOWN | 12-05-2025 16:42:40 | 2d 23h 50m 53s | 3/3 | check_ifoperstatus v2.4.0 (monitoring-plugins 2.4.0) |
| | Ping | OK | 12-05-2025 16:33:34 | 0d 23h 20m 9s | 1/3 | PING OK - Packet loss = 0%, RTA = 1.11 ms |

***Remonter les logs de HaProxy sur rsyslog et commenter les résultats en fonction des tests réalisé**

On installe rsyslog sur haproxy.

```
root@Haproxy:~# apt install rsyslog -y
```

On active la **journalisation** dans HAProxy. Les facilités **local0** et **local1** sont des options de **syslog**. Cela permet à HAProxy d'envoyer ses logs vers **rsyslog**.

```
global
        log /dev/log    local0
        log /dev/log    local1 notice
```

`option httplog` → permet d'obtenir un **format détaillé des requêtes HTTP** dans les logs.

```
defaults
        log     global
        mode    http
        option  httplog
```

On crée ce fichier. Tous les logs de **HAProxy** utilisant la facilité **local0** seront écrits dans **`/var/log/haproxy.log`**.

```
root@Haproxy:~# vim /etc/rsyslog.d/haproxy.conf
```

```
sio21@SRVmdb2: ~   sio21@mariadb: ~   LOG   sio@Haproxy: ~
# Fichier rsyslog pour HAProxy
*.*     /var/log/haproxy.log
*.*     /var/log/haproxy_admin.log

if ($fromhost-ip == 'IP_HAPROXY') then /var/log/haproxy-central.log
& stop
```

On redémarre les deux services, puis on effectue un test pour afficher les logs. On constate que les logs de **HAProxy** remontent correctement sur **rsyslog** dans **/var/log/haproxy/haproxy12-26.log**.

```
root@SrvRsyslog:~# tail -f  /var/log/haproxy/haproxy12-26.log
Dec  5 15:58:35 Haproxy sshd-session[46756]:  Connection closed by 127.0.0.1 port 47676 [pre
auth]
Dec  5 15:58:35 Haproxy rsyslogd:  omfwd: [wrkr 0/140148580538048] we had a generic or IO er
ror with the remote server. The actual error message should already have been provided.  Ser
ver is 192.168.13.5:514. This can be caused by the remote server or an interim system like a
 load balancer or firewall. Rsyslog will re-open the connection if configured to do so. [v8.
2504.0 try https://www.rsyslog.com/e/2027 ]
Dec  5 15:58:35 Haproxy rsyslogd:  omfwd: [wrkr 0] target 192.168.13.5:514 became unavailabl
e during buffer flush. Remaining messages will be sent when it is online again. [v8.2504.0 t
ry https://www.rsyslog.com/e/2007 ]
Dec  5 15:58:35 Haproxy rsyslogd:  omfwd: [wrkr 0/140148580538048] no working target servers
 in pool available, suspending action [v8.2504.0 try https://www.rsyslog.com/e/2007 ]
Dec  5 15:58:35 Haproxy rsyslogd:  action 'action-12-builtin:omfwd' suspended (module 'built
in:omfwd'), retry 0. There should be messages before this one giving the reason for suspensi
on. [v8.2504.0 try https://www.rsyslog.com/e/2007 ]
Dec  5 15:58:35 Haproxy rsyslogd:  action 'action-12-builtin:omfwd' resumed (module 'builtin
:omfwd') [v8.2504.0 try https://www.rsyslog.com/e/2359 ]
Dec  5 15:58:43 192.168.12.26 haproxy[46674]:  192.168.12.250:33728 [05/Dec/2025:15:58:43.96
7] stats stats/<STATS> 0/0/0/0/0 200 26816 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTTP/1.1"
Dec  5 15:58:54 192.168.12.26 haproxy[46674]:  192.168.12.250:33728 [05/Dec/2025:15:58:54.05
9] stats stats/<STATS> 0/0/0/0/0 200 26820 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTTP/1.1"
Dec  5 15:59:04 192.168.12.26 haproxy[46674]:  192.168.12.250:33728 [05/Dec/2025:15:59:04.14
6] stats stats/<STATS> 0/0/0/0/0 200 26820 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTTP/1.1"
Dec  5 15:59:15 192.168.12.26 haproxy[46674]:  192.168.12.250:33728 [05/Dec/2025:15:59:15.33
```

La capture ci-dessous montre le changement des sites lorsqu'on actualise la page.

```
auth]
Dec  5 16:03:35 192.168.12.26 haproxy[46674]:  192.168.12.251:56882 [05/Dec/2025:16:03:35.92
1] http_front http_back/web2 0/0/0/1/1 200 273 - - ---- 2/1/0/0/0 0/0 "GET / HTTP/1.1"
Dec  5 16:03:36 192.168.12.26 haproxy[46674]:  192.168.12.251:56882 [05/Dec/2025:16:03:36.87
9] http_front http_back/web3 0/0/0/1/1 200 273 - - ---- 2/1/0/0/0 0/0 "GET / HTTP/1.1"
Dec  5 16:03:37 192.168.12.26 haproxy[46674]:  192.168.12.251:56882 [05/Dec/2025:16:03:37.51
2] http_front http_back/web1 0/0/1/1/2 200 273 - - ---- 2/1/0/0/0 0/0 "GET / HTTP/1.1"
```

Ces logs montrent deux types d'événements. D'abord, rsyslog sur le serveur HAProxy rencontre **des difficultés pour envoyer les logs vers le serveur Rsyslog central** (192.168.13.5:514). Le système suspend temporairement l'envoi, puis reprend automatiquement dès que la connexion est rétablie. Cela signifie que la centralisation des logs est configurée mais que la communication réseau ou la disponibilité du serveur central peut poser problème.

Ensuite, on voit des **logs normaux générés par HAProxy** indiquant le trafic HTTP traité par le load balancer. Les lignes contiennent l'IP du client, l'heure de la requête, l'endpoint demandé (ici la page de stats HAProxy), le code HTTP renvoyé et des informations de performance comme le temps de traitement et le nombre de connexions.En résumé, les logs montrent à la fois **les problèmes de transmission vers Rsyslog** et **le fonctionnement normal de HAProxy**, ce qui permet de vérifier à la fois la centralisation et le suivi du trafic.

## Mission 3 : Mise en place d'un outil de haute disponibilité avec Heartbeat

On clone le serveur **HAProxy** en modifiant son **hostname**. Le nouveau serveur sera nommé **haproxy2**.

```
root@Haproxy2:~#
```

Son ip est 192.168.12.20.

```
ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether bc:24:11:59:32:cb brd ff:ff:ff:ff:ff:ff
 altname enp0s18
 altname enxbc24115932cb
 inet 192.168.12.28/24 brd 192.168.12.255 scope global ens18
    valid_lft forever preferred_lft forever
 inet6 fe80::be24:11ff:fe59:32cb/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
```

On la mets donc dans le Vlan DMZ.

```
Network Device (net0)        virtio=BC:24:11:59:32:CB,bridge=vmbr0,tag=452
```

On procède au **référencement DNS**.

```
Haproxy2        IN      A       192.168.12.28
~
```

On effectue une vérification DNS.

```
root@clonedns:~# nslookup Haproxy2
Server:         192.168.12.1
Address:        192.168.12.1#53

Name:   Haproxy2.menuimetal.fr
Address: 192.168.12.28
```

On remonte le deuxième serveur sur **Nagios** en refaisant exactement les mêmes commandes, en modifiant simplement le **nom du serveur**.

| | | | | | | |
|---|---|---|---|---|---|---|
| Haproxy2 | CPU Load | OK | 12-05-2025 11:27:15 | 0d 0h 34m 27s | 1/3 | LOAD |
| | DISK | OK | 12-05-2025 11:31:07 | 0d 0h 30m 35s | 1/3 | DISK O |
| | HAProxy Service | OK | 12-05-2025 11:24:51 | 0d 21h 37m 0s | 1/3 | PROCS |
| | HAProxy Stats | UNKNOWN | 12-05-2025 11:27:32 | 0d 21h 31m 41s | 3/3 | Usage: |
| | Interface eth0 Status | UNKNOWN | 12-05-2025 11:21:54 | 0d 0h 9m 48s | 3/3 | check_i |
| | Ping | OK | 12-05-2025 11:22:10 | 0d 0h 49m 36s | 1/3 | PING O |

***Installer le service Heartbeat sur les deux serveurs**

```
root@Haproxy2:~# apt install heartbeat
```

On vérifie donc l'installation sur nos deux serveurs **HAProxy** à l'aide de la commande dpkg.

```
root@Haproxy:~# dpkg -l | grep heartbeat
ii  heartbeat                          1:3.0.6-17
       amd64        Subsystem for High-Availability Linux
ii  libheartbeat2t64                  1:3.0.6-17
       amd64        Subsystem for High-Availability Linux (libraries)
```

```
root@Haproxy2:~# dpkg -l | grep heartbeat
ii  heartbeat                          1:3.0.6-17
       amd64        Subsystem for High-Availability Linux
ii  libheartbeat2t64                  1:3.0.6-17
       amd64        Subsystem for High-Availability Linux (libraries)
```

***Configurer les fichiers ha.cf et haresources et expliquer leur rôle**

On crée le dossier sur les deux serveurs.

```
root@Haproxy:~# mkdir -p /etc/ha.d
root@Haproxy:~#
```

```
root@Haproxy2:~# mkdir -p /etc/ha.d
root@Haproxy2:~#
```

On crée un fichier **ha.cf** et on y ajoute le contenu indiqué. Cette opération sera réalisée sur les **deux serveurs**.

```
192.168.12.28
192.168.12.28   192.168.12.26
GNU nano 8.4                    /etc/ha.d/ha.cf *
logfacility     local0
keepalive       2
deadtime        10
warntime        5
initdead        20
bcast           ens18
node            Haproxy
node            Haproxy2
auto_failback   on
```

```
192.168.12.26
192.168.12.28   192.168.12.26
GNU nano 8.4                    /etc/ha.d/ha.cf *
logfacility     local0
keepalive       2
deadtime        10
warntime        5
initdead        20
bcast           ens18
node            Haproxy
node            Haproxy2
auto_failback   on
```

Le fichier définit comment les 2 serveurs Heartbeat communiquent :

- **keepalive** : fréquence d'envoi des paquets « je suis vivant »
- **deadtime** : temps avant de considérer un nœud comme mort
- **bcast eth0** : communication par broadcast sur l'interface eth0
- **node** : nom des serveurs dans le cluster
- **auto_failback** : retour automatique de la ressource sur le nœud maître

J'ai également créé un fichier `authkeys` pour **définir les clés d'authentification** entre les serveurs HA afin de sécuriser la communication et la synchronisation des informations.

```
192.168.12.26
192.168.12.28   192.168.12.26
GNU nano 8.4                    /etc/ha.d/authkeys *
auth 1
1 sha1 sio21
```

```
192.168.12.28
192.168.12.28   192.168.12.26
GNU nano 8.4                    /etc/ha.d/authkeys *
auth 1
1 sha1 sio21
```

Le fichier `/etc/ha.d/authkeys` sert à **sécuriser la communication entre les nœuds Heartbeat**.

Heartbeat utilise ce fichier pour **authentifier chaque paquet envoyé entre les serveurs** du cluster.
Sans ce fichier, ou si les mots de passe ne correspondent pas, les nœuds ne pourront pas se faire confiance et **le cluster ne fonctionnera pas**.

maintenant nous créons le fichiers haresources et on y ajoute l ip ce qui est fait sur les deux serveur signifie :

1. Haproxy → le nœud maître par défaut. C'est le serveur qui prendra initialement la VIP et lancera le service HAProxy.

2. 192.168.12.100 → la VIP (Virtual IP) qui sera basculée automatiquement entre les nœuds en cas de panne.

3. haproxy → le service à gérer, c'est-à-dire le processus HAProxy à démarrer/arrêter selon le nœud actif

```
192.168.12.28
192.168.12.26   192.168.12.28
GNU nano 8.4                    /etc/ha.d/haresources
Haproxy IPaddr::192.168.12.100/24/ens18 haproxy
```

```
192.168.12.26
192.168.12.26   192.168.12.28
GNU nano 8.4                    /etc/ha.d/haresources
Haproxy IPaddr::192.168.12.100/24/ens18 haproxy
```

On attribue les droits appropriés au fichier **`authkeys`** sur les deux serveurs, puis on effectue un test pour vérifier que tout est correctement activé.

```
root@Haproxy2:~# chmod 600 /etc/ha.d/authkeys
root@Haproxy2:~# ls -l /etc/ha.d/authkeys
-rw------- 1 root root 20  4 déc.  14:10 /etc/ha.d/authkeys
root@Haproxy2:~# systemctl restart heartbeat
root@Haproxy2:~# systemctl status heartbeat
● heartbeat.service - Heartbeat High Availability Cluster Communication and Membership
     Loaded: loaded (/usr/lib/systemd/system/heartbeat.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-04 14:28:22 CET; 15s ago
```

Le **ping** entre les deux serveurs fonctionne correctement, ce qui confirme une bonne communication.

```
    link/ether bc:24:11:36:73:d2 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24113673d2
    inet 192.168.12.26/24 brd 192.168.12.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe36:73d2/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
root@Haproxy:~# ping 192.168.12.28
PING 192.168.12.28 (192.168.12.28) 56(84) bytes of data.
64 bytes from 192.168.12.28: icmp_seq=1 ttl=64 time=1.21 ms
64 bytes from 192.168.12.28: icmp_seq=2 ttl=64 time=0.569 ms
64 bytes from 192.168.12.28: icmp_seq=3 ttl=64 time=0.370 ms
^C
--- 192.168.12.28 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.370/0.716/1.210/0.358 ms
```

```
pipe 3
root@Haproxy2:~# ping 192.168.12.26
PING 192.168.12.26 (192.168.12.26) 56(84) bytes of data.
64 bytes from 192.168.12.26: icmp_seq=1 ttl=64 time=0.312 ms
64 bytes from 192.168.12.26: icmp_seq=2 ttl=64 time=0.528 ms
64 bytes from 192.168.12.26: icmp_seq=3 ttl=64 time=0.642 ms
64 bytes from 192.168.12.26: icmp_seq=4 ttl=64 time=0.270 ms
^X64 bytes from 192.168.12.26: icmp_seq=5 ttl=64 time=0.313 ms
^C
--- 192.168.12.26 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4082ms
rtt min/avg/max/mdev = 0.270/0.413/0.642/0.145 ms
root@Haproxy2:~#
```

Je fais un `systemctl restart heartbeat` sur les **deux serveurs**.

```
root@Haproxy:~# systemctl restart heartbeat
```

On vérifie ensuite le **status** et tout est bien activé pour **HAProxy**.

```
root@Haproxy:~# systemctl status  heartbeat
● heartbeat.service - Heartbeat High Availability Cluster Communication and Membership
     Loaded: loaded (/usr/lib/systemd/system/heartbeat.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-04 16:05:39 CET; 3min 16s ago
 Invocation: 242474df05c84c1590380cf6e29a522b
       Docs: man:heartbeat(8)
```

Pour Haproxy2.

```
root@Haproxy2:~# systemctl status heartbeat
● heartbeat.service - Heartbeat High Availability Cluster Communication and Membership
     Loaded: loaded (/usr/lib/systemd/system/heartbeat.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-04 16:05:15 CET; 27s ago
 Invocation: e33274e576b04a069d55646e744adbc9
       Docs: man:heartbeat(8)
             http://www.linux-ha.org/wiki/Documentation
```

On exécute la commande `ip a` : la **VIP 192.168.12.100** est bien active sur le nœud **HAProxy**. Le cluster **Heartbeat** fonctionne correctement : si le nœud maître tombe, Heartbeat pourra basculer cette VIP sur **haproxy2**.

```
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
qlen 1000
    link/ether bc:24:11:36:73:d2 brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24113673d2
    inet 192.168.12.26/24 brd 192.168.12.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet 192.168.12.100/24 brd 192.168.12.255 scope global secondary ens18:0
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe36:73d2/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

La **VIP** n'est pas active sur **haproxy2**, ce qui est normal tant que le nœud maître est actif. Heartbeat configure la VIP uniquement sur le maître ; elle bascule sur le deuxième nœud seulement si le maître tombe ou si Heartbeat est arrêté sur le maître.

```
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
qlen 1000
    link/ether bc:24:11:59:32:cb brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24115932cb
    inet 192.168.12.28/24 brd 192.168.12.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe59:32cb/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

On va tester le **failover** : pour cela, on arrête **Heartbeat** sur le serveur maître **haproxy**.

```
root@Haproxy:~# systemctl stop heartbeat
```

Donc le heartbeat est bien inactive.

```
root@Haproxy:~# systemctl status heartbeat
○ heartbeat.service - Heartbeat High Availability Cluster Communication and Membership
     Loaded: loaded (/usr/lib/systemd/system/heartbeat.service; enabled; preset: enabled)
     Active: inactive (dead) since Thu 2025-12-04 16:17:30 CET; 17s ago
   Duration: 11min 48.066s
 Invocation: 242474df05c84c1590380cf6e29a522b
```

Ensuite, sur **haproxy2**, on exécute `ip a` pour vérifier que la **VIP** est bien activée. C'est le cas, ce qui confirme le bon fonctionnement du **failover**.

```
root@Haproxy2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default
qlen 1000
    link/ether bc:24:11:59:32:cb brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    altname enxbc24115932cb
    inet 192.168.12.28/24 brd 192.168.12.255 scope global ens18
       valid_lft forever preferred_lft forever
    inet 192.168.12.100/24 brd 192.168.12.255 scope global secondary ens18:0
       valid_lft forever preferred_lft forever
    inet6 fe80::be24:11ff:fe59:32cb/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

On vérifie que **HAProxy** est bien actif sur le nouveau maître, **haproxy2**.

```
root@Haproxy2:~# systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-04 16:17:20 CET; 1min 59s ago
 Invocation: 5708a637353342dca85388f022211bae
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
   Main PID: 28936 (haproxy)
     Status: "Ready."
      Tasks: 2 (limit: 2303)
     Memory: 58.6M (peak: 59.2M)
        CPU: 159ms
     CGroup: /system.slice/haproxy.service
             ├─28936 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid>
             └─28938 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid>

déc. 04 16:17:20 Haproxy2 systemd[1]: Starting haproxy.service - HAProxy Load Balancer...
déc. 04 16:17:20 Haproxy2 haproxy[28936]: [NOTICE]   (28936) : New worker (28938) forked
```

Pour revenir au comportement normal, on redémarre **Heartbeat** sur **haproxy** et la **VIP** est récupérée automatiquement.

```
root@Haproxy:~# ip a | grep 192.168.12.100
    inet 192.168.12.100/24 brd 192.168.12.255 scope global secondary ens18:0
```

On continue avec un test dans notre navigateur pour vérifier que l'IP **192.168.12.100** est bien configurée et redirige correctement vers nos pages web. Le test fonctionne pour les trois serveurs même en arrêtant **haproxy2**. L'IP **192.168.12.100** correspond à notre **VIP**.

💡 **Règle d'or** : la VIP n'apparaît jamais simultanément sur les deux nœuds sauf si tu fais du multicast très spécifique (rare et dangereux). Heartbeat gère toujours **un seul maître à la fois**.

***Remonter les logs du service Heartbeat sur rsyslog et commenter les résultats en fonction des tests réalisés**

Je me connecte à mon serveur **rsyslog**, je crée le fichier `remote.conf` et j'y ajoute la configuration suivante :

```
192.168.12.26    192.168.12.28    192.168.13.5
  GNU nano 8.4                    /etc/rsyslog.d/remote.conf *
# Activer la réception UDP
module(load="imudp")
input(type="imudp" port="514")

# Activer la réception TCP
module(load="imtcp")
input(type="imtcp" port="514")

# Stocker les logs Heartbeat dans un fichier séparé
if $fromhost-ip == '192.168.12.26' or $fromhost-ip == '192.168.12.28' then /var/log/heartb>
& stop
```

Je fais ensuite un restart de rsyslog

```
root@SrvRsyslog:~# systemctl restart rsyslog
root@SrvRsyslog:~#
```

Ensuite, je vérifie le **status** et tout est bien actif.

```
root@SrvRsyslog:~# systemctl status rsyslog
● rsyslog.service - System Logging Service
     Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-04 16:45:52 CET; 38s ago
 Invocation: d89d7ab500eb40b1a96c10bfb03ec566
TriggeredBy: ● syslog.socket
```

Je vérifie sur quel port le serveur écoute : il est bien configuré pour **UDP** et **TCP 514**.

```
root@SrvRsyslog:~# ss -tulpn | grep 514
udp   UNCONN 0      0            0.0.0.0:514        0.0.0.0:*    users:(("rsyslogd",pid=123
543,fd=8))

udp   UNCONN 0      0            0.0.0.0:514        0.0.0.0:*    users:(("rsyslogd",pid=123
543,fd=6))
```

Maintenant, on va configurer les serveurs **haproxy** et **haproxy2**. Dans le fichier `rsyslog.conf`, on ajoute la dernière ligne de configuration sur **les deux serveurs**.

```
192.168.12.26   192.168.12.28   192.168.13.5   192.168.12.20
  GNU nano 8.4                    /etc/rsyslog.conf *
# /etc/rsyslog.conf configuration file for rsyslog
#
# For more information install rsyslog-doc and see
# /usr/share/doc/rsyslog-doc/html/configuration/index.html


#################
#### MODULES ####
#################

# Envoyer tous les logs Heartbeat et HAProxy vers le serveur rsyslog central
local0.* @@192.168.13.5:514    # @@ = TCP, @ = UDP
```

Ensuite, on redémarre le service **Heartbeat** pour que les changements soient pris en compte.

```
root@Haproxy2:~# systemctl restart heartbeat
root@Haproxy2:~#
```

Sur le serveur **rsyslog**, on crée un fichier `heartbeat.conf` et on y ajoute ces commandes afin de pouvoir suivre séparément les Heartbeat de chaque serveur, tout en conservant la possibilité d'avoir un fichier global si nécessaire.

```
192.168.12.26   192.168.12.28   192.168.13.5   192.168.12.20
  GNU nano 8.4                /etc/rsyslog.d/heartbeat.conf
# Logs Heartbeat depuis les nœuds HAProxy
local0.*    /var/log/heartbeat.log
# Séparer les logs Heartbeat de chaque serveur
if $fromhost-ip == '192.168.12.26' then /var/log/heartbeat-haproxy1.log
& stop

if $fromhost-ip == '192.168.12.28' then /var/log/heartbeat-haproxy2.log
& stop
```

Puis on attribue les droit.

```
root@SrvRsyslog:~#  touch /var/log/heartbeat.log
root@SrvRsyslog:~# chown root:adm /var/log/heartbeat.log
root@SrvRsyslog:~# chmod 640 /var/log/heartbeat.log
```

On redémarre tous les services, puis on consulte les logs pour HAProxy1 et HAProxy2 à l'aide de la commande :tail -f /var/log/heartbeat-haproxy1.log /var/log/heartbeat-haproxy2.log
HAProxy1

- HAProxy1 est le nœud actif du cluster.

- Il gère directement le trafic HTTP et répond aux requêtes des utilisateurs.

- Dans les logs, on peut observer les accès aux différentes ressources (stats/<STATS>, http_front, http_back/web1), ce qui indique que HAProxy1 traite normalement les requêtes.

- Les événements Heartbeat présents dans /var/log/heartbeat-haproxy1.log montrent que ce nœud possède les ressources principales du cluster et qu'il est pleinement opérationnel.

HAProxy2

- HAProxy2 joue le rôle de nœud secondaire ou standby.

- Il n'intervient pas dans le traitement du trafic tant que HAProxy1 est actif.

- Les logs situés dans /var/log/heartbeat-haproxy2.log montrent que HAProxy2 a acquis les ressources en standby et attend une éventuelle bascule si HAProxy1 venait à tomber.

- Ces messages permettent de vérifier que le nœud est prêt et que la redondance fonctionne correctement.

```
^C
root@SrvRsyslog:~# tail -f /var/log/heartbeat-haproxy1.log /var/log/heartbeat-haproxy2.log
==> /var/log/heartbeat-haproxy1.log <==
2025-12-05T14:58:45+01:00 Haproxy heartbeat: [44212]: info: No pkts missing from haproxy2!
2025-12-05T14:58:45+01:00 Haproxy heartbeat: [44212]: info: Other node completed standby ta
keover of foreign resources.
2025-12-05T14:58:52+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:58:52.244] stats stats/<STATS> 0/0/0/0/0 200 26498 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T14:59:02+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:59:02.356] stats stats/<STATS> 0/0/0/0/0 200 26498 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T14:59:13+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:59:13.551] stats stats/<STATS> 0/0/0/0/0 200 26498 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T14:59:23+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:59:23.685] stats stats/<STATS> 0/0/0/0/0 200 26493 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T14:59:33+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:59:33.755] stats stats/<STATS> 0/0/0/0/0 200 26498 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T14:59:44+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:59:44.186] stats stats/<STATS> 0/0/0/0/0 200 26496 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T14:59:54+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
4:59:54.290] stats stats/<STATS> 0/0/0/0/0 200 26498 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T15:00:04+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
5:00:04.528] stats stats/<STATS> 0/0/0/0/0 200 26498 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"

==> /var/log/heartbeat-haproxy2.log <==
2025-12-05T14:58:37+01:00 Haproxy2 heartbeat: [32386]: info: remote resource transition com
pleted.
2025-12-05T14:58:37+01:00 Haproxy2 heartbeat: [32386]: info: remote resource transition com
pleted.
2025-12-05T14:58:37+01:00 Haproxy2 heartbeat: [32386]: info: Local Resource acquisition com
pleted. (none)
2025-12-05T14:58:37+01:00 Haproxy2 heartbeat: [32386]: info: haproxy wants to go standby [f
oreign]
2025-12-05T14:58:38+01:00 Haproxy2 heartbeat: [32386]: info: standby: acquire [foreign] res
ources from haproxy
2025-12-05T14:58:38+01:00 Haproxy2 heartbeat: [32414]: info: acquire local HA resources (st
andby)
```

**Heartbeat**

- Heartbeat est le composant qui **supervise le cluster HA**.
- Il coordonne la **distribution des ressources** entre les nœuds et assure la **haute disponibilité** en cas de panne d'un nœud actif.
- Les logs Heartbeat indiquent les **transitions de ressources** (`remote resource transition completed`, `standby resource acquisition done`) et l'**état actuel de chaque nœud**.
- Ces informations permettent de suivre en **temps réel la santé du cluster** et de s'assurer que la **bascule automatique** fonctionnera correctement si nécessaire.

```
2025-12-05T15:01:22+01:00 Haproxy rsyslogd: action 'action-12-builtin:omfwd' suspended (mod
ule 'builtin:omfwd'), retry 0. There should be messages before this one giving the reason f
or suspension. [v8.2504.0 try https://www.rsyslog.com/e/2007 ]
2025-12-05T15:01:22+01:00 Haproxy pacemaker-controld[13383]:  notice: Transition 101 (Compl
ete=0, Pending=0, Fired=0, Skipped=0, Incomplete=0, Source=/var/lib/pacemaker/pengine/pe-er
ror-0.bz2): Complete
2025-12-05T15:01:22+01:00 Haproxy pacemaker-controld[13383]:  notice: State transition S_TR
ANSITION_ENGINE -> S_IDLE
2025-12-05T15:01:22+01:00 Haproxy rsyslogd: action 'action-12-builtin:omfwd' resumed (modul
e 'builtin:omfwd') [v8.2504.0 try https://www.rsyslog.com/e/2359 ]
2025-12-05T15:01:26+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
5:01:26.425] stats stats/<STATS> 0/0/0/0/0 200 26602 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T15:01:36+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
5:01:36.555] stats stats/<STATS> 0/0/0/0/0 200 26602 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T15:01:42+01:00 192.168.12.26 haproxy[45731]: 127.0.0.1:33862 [05/Dec/2025:15:01:
42.250] http_front http_back/web2 0/0/0/1/1 200 273 - - ---- 2/1/0/0/0 0/0 "GET / HTTP/1.1"
2025-12-05T15:01:46+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
5:01:46.655] stats stats/<STATS> 0/0/0/0/0 200 26605 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
2025-12-05T15:01:56+01:00 192.168.12.26 haproxy[45731]: 192.168.12.250:46314 [05/Dec/2025:1
5:01:56.737] stats stats/<STATS> 0/0/0/0/0 200 26608 - - LR-- 1/1/0/0/0 0/0 "GET /stats HTT
P/1.1"
```

- **HAProxy1** est actif, traite le trafic HTTP et maintient les ressources principales.
- **HAProxy2** est en standby, prêt à prendre le relais si HAProxy1 tombe.
- **Heartbeat** fonctionne correctement entre les deux nœuds, avec les transitions de ressources bien enregistrées.

***Surveiller les nœuds Heartbeat via Nagios**

On se rend sur notre serveur **Nagios** et on crée un fichier nommé `check_heartbeat.sh`.

```
  GNU nano 8.4        /usr/lib/nagios/plugins/check_heartbeat.sh *
#!/bin/bash

HOST=$1

STATUS=$(ssh -o BatchMode=yes $HOST 'systemctl is-active heartbeat' 2>/dev/null)

if [ "$STATUS" = "active" ]; then
    echo "OK - Heartbeat est actif sur $HOST"
    exit 0
elif [ "$STATUS" = "inactive" ]; then
    echo "CRITICAL - Heartbeat est INACTIF sur $HOST"
    exit 2
else
    echo "UNKNOWN - Statut Heartbeat : $STATUS"
    exit 3
fi
```

On attribue les droits au fichier à l'aide de `chmod`.

```
root@Nagios1:~# nano /usr/lib/nagios/plugins/check_heartbeat.sh
root@Nagios1:~# chmod +x /usr/lib/nagios/plugins/check_heartbeat.sh
root@Nagios1:~#
```

Je me rends dans le fichier `commands.cfg` et j'ajoute la partie **Heartbeat TCP**.

```
define command{
    command_name    check_heartbeat_tcp
    command_line    /usr/lib/nagios/plugins/check_tcp -H $HOSTADDRESS$ -p $ARG1$
}
```

Puis, dans mon fichier `localhost.cfg`, j'ajoute le `define service` correspondant à la surveillance de Heartbeat.

```
##########heartbeat

define service{
        use                     generic-service
        host_name               Haproxy,Haproxy2
        service_description     Heartbeat Status
        check_command           check_tcp!8080
}
```

On effectue le test avec la commande :`nagios4 -v /etc/nagios4/nagios.cfg` afin de s'assurer qu'il n'y a **aucune erreur** dans la configuration.

```
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0
```

Puis on redemarre Nagios.

```
root@Nagios1:~# systemctl restart nagios4
root@Nagios1:~#
```

On se rend sur l'interface **Nagios** et on constate que le service a bien été ajouté et que tout fonctionne correctement.

| | | | | | | |
|---|---|---|---|---|---|---|
| Haproxy | CPU Load | OK | 12-05-2025 16:41:10 | 0d 5h 47m 35s | 1/3 | LOAD OK - total load average: 0.00, 0.01, 0.00 |
| | DISK | OK | 12-05-2025 16:39:51 | 0d 5h 43m 42s | 1/3 | DISK OK - free space: / 26904MiB (91,2% inode=96%): |
| | HAProxy Service | OK | 12-05-2025 16:37:22 | 2d 23h 56m 11s | 1/3 | PROCS OK: 104 processes |
| | HAProxy Stats | UNKNOWN | 12-05-2025 16:41:28 | 0d 0h 2m 5s | 3/3 | Usage: check_haproxy [ -v |
| | Heartbeat Status | OK | 12-05-2025 16:36:06 | 0d 2h 27m 27s | 1/3 | TCP OK - 0,001 second response time on 192.168.12.26 port 8080 |
| | Interface eth0 Status | UNKNOWN | 12-05-2025 16:42:40 | 2d 23h 50m 53s | 3/3 | check_ifoperstatus v2.4.0 (monitoring-plugins 2.4.0) |
| | Ping | OK | 12-05-2025 16:33:34 | 0d 23h 20m 9s | 1/3 | PING OK - Packet loss = 0%, RTA = 1.11 ms |
| Haproxy2 | CPU Load | OK | 12-05-2025 16:42:15 | 0d 5h 46m 18s | 1/3 | LOAD OK - total load average: 0.00, 0.01, 0.00 |
| | DISK | OK | 12-05-2025 16:41:10 | 0d 5h 42m 26s | 1/3 | DISK OK - free space: / 26904MiB (91,2% inode=96%): |
| | HAProxy Service | OK | 12-05-2025 16:34:51 | 1d 2h 48m 51s | 1/3 | PROCS OK: 105 processes |
| | HAProxy Stats | UNKNOWN | 12-05-2025 16:42:37 | 1d 2h 43m 32s | 3/3 | Usage: check_haproxy [ -v |
| | Heartbeat Status | OK | 12-05-2025 16:41:19 | 0d 1h 22m 14s | 1/3 | TCP OK - 0,001 second response time on 192.168.12.28 port 8080 |
| | Interface eth0 Status | UNKNOWN | 12-05-2025 16:41:54 | 0d 5h 21m 39s | 3/3 | check_ifoperstatus v2.4.0 (monitoring-plugins 2.4.0) |
| | Ping | OK | 12-05-2025 16:42:10 | 0d 6h 1m 27s | 1/3 | PING OK - Packet loss = 0%, RTA = 1.07 ms |

J'ai également ajouté une section pour indiquer que l'**IP virtuelle 192.168.12.100** est surveillée. On peut vérifier son état directement en ligne de commande.

```
root@Nagios1:~# sudo -u nagios /usr/lib/nagios/plugins/check_tcp -H 192.168.12.100
 -p 8080
TCP OK - 0,001 second response time on 192.168.12.100 port 8080|time=0,001084s;;;0
,000000;10,000000
```

Sur l'interface, l'état est bien **vert**, ce qui indique que tout fonctionne correctement.

| | | | |
|---|---|---|---|
| HAProxy-VIP | Heartbeat Status | OK | 12-05-2025 16:23:38 |

Notes :**HAProxy** est un répartiteur de charge (load balancer) qui distribue le trafic entre plusieurs serveurs, et **Heartbeat** est un service de haute disponibilité qui surveille les nœuds et assure le basculement automatique en cas de panne.

////////Problème Nagios//////////////

On effectue un **test pour vérifier si les statistiques sur Nagios fonctionnent**.

- En **ligne de commande**, tout fonctionne correctement pour **HAProxy1**, dont l'état est **OK**, contrairement à ce que l'interface pourrait indiquer.

- On remarque qu'à la différence de la commande, l'**IP de la VM** change lors du test.

- Pour la deuxième capture, il s'agit de **HAProxy2**, également surveillé et prêt à prendre le relais si nécessaire.

```
root@Nagios1:~# /usr/lib/nagios/plugins/check_haproxy -u "http://admin:password@19
2.168.12.26:8080/stats;csv" -w 80 -c 90
HAPROXY OK -  http_back (Active: 3/3) | t=0.063125s;80;90;0; sess_http_back=0;;;0;
26212 bytes_in_http_back=0B;;;0; bytes_out_http_back=0B;;;0;
root@Nagios1:~#
```

```
root@Nagios1:~# /usr/lib/nagios/plugins/check_haproxy -u "http://admin:password@19
2.168.12.28:8080/stats;csv" -w 80 -c 90
HAPROXY OK -  http_back (Active: 3/3) | t=0.063319s;80;90;0; sess_http_back=0;;;0;
26212 bytes_in_http_back=0B;;;0; bytes_out_http_back=0B;;;0;
root@Nagios1:~#
```